

Ням Шеф: ИИ-консультант — передача бэкенду

Oct 6, 2026 · @Ekaterina

Модуль кулинарного консультанта на Python готов и проверен локально: FastAPI-эндпоинт принимает сообщение с историей диалога и возвращает ответ модели Qwen3.5-9B. Сейчас модель работает через LM Studio на ноутбуке; для работы 24/7 достаточно переключить провайдера в `.env` на OpenRouter или Together AI, файлы с кодом править не нужно: настройки всех трёх провайдеров уже записаны в `chef_llm_client.py`, а `.env` только выбирает, какой из них использовать.

1. Какие файлы передаются

Передаются пять файлов. Все кладутся в одну папку, например `backend/ai/`.

Файл	Что внутри	Кто правит
<code>chef_llm_client.py</code>	Клиент к модели: системный промпт, параметры генерации, обрезка истории до 10 сообщений, выбор провайдера, обработка ошибок	Промпт и параметры — ML (Катя); остальное — бэкенд
<code>main.py</code>	FastAPI-приложение с эндпоинтами <code>POST /api/v1/chat</code> и <code>GET /api/v1/health</code>	Бэкенд
<code>requirements.txt</code>	Зависимости: <code>fastapi</code> , <code>uvicorn</code> , <code>openai</code> , <code>pydantic</code> , <code>python-dotenv</code>	Бэкенд
<code>.env.example</code>	Образец настроек без секретов; из него делается свой <code>.env</code>	Бэкенд
<code>test_lm.py</code>	Отдельный скрипт для ручной проверки модели в консоли, без бэкенда	ML

Не передаём и не кладём в git: файл `.env` — в нём будет API-ключ облачного провайдера, то есть доступ к счёту. В `.gitignore` добавить строку `.env`.

2. Как это устроено

Телефон никогда не обращается к модели напрямую: всё идёт через бэкенд. Путь одного сообщения:

1. Приложение отправляет `POST /api/v1/chat`: новое сообщение пользователя и историю предыдущих сообщений.
2. FastAPI проверяет запрос: сообщение не пустое, не длиннее 2 000 символов, история не длиннее 50 элементов. Неверный запрос получает ответ 422.
3. Шлюз вежливости `is_polite` проверяет текст (пока не подключён, см. раздел 6). Грубое сообщение получает вежливую заглушку сразу, модель не вызывается.
4. `ChefLlmClient` собирает запрос к модели: системный промпт + 10 последних сообщений истории + новый вопрос.
5. Запрос уходит провайдеру из `.env`: LM Studio на ноутбуке, OpenRouter или Together AI. Формат запроса у всех одинаковый (OpenAI-совместимый API).
6. Клиент вырезает из ответа блок рассуждений модели `<think>...</think>` и возвращает приложению чистый текст в поле `reply`.

Почему так: промпт, ключ API и логика обрезки истории живут только на сервере. Их можно менять без обновления приложения, а ключ не попадает на телефоны пользователей.

3. Как запустить у себя

Нужны Python 3.10+ и LM Studio с моделью `qwen/qwen3.5-9b` (файл `Qwen3.5-9B-Q4_K_M.gguf`, ~6,5 ГБ). Если LM Studio ставить не хочется, можно сразу работать с облаком (раздел 5) — тогда нужен только ключ.

1. В LM Studio: вкладка **Developer** → загрузить модель → статус сервера **Running**. Длина контекста модели (Context Length) — 8192.
2. В папке с файлами создать `.env`:

```
LLM_PROVIDER=lmstudio
LLM_MODEL=qwen/qwen3.5-9b
```

3. Установить зависимости: `pip install -r requirements.txt`.

4. Запустить: `python -m uvicorn main:app --host 0.0.0.0 --port 8000`. Флаг `--host 0.0.0.0` нужен, чтобы к бэкенду мог обратиться телефон в той же Wi-Fi сети.
5. Проверить связь с моделью: `http://localhost:8000/api/v1/health` должен вернуть `"llm": true`.
6. Отправить тестовое сообщение со страницы `http://localhost:8000/docs` (POST `/api/v1/chat` → Try it out → Execute).

На ноутбуке с RTX 3050 4 ГБ ответ занимает 30–45 секунд — это нормально для локальной модели. В облаке ответ приходит за несколько секунд.

4. API для мобильного приложения

Один эндпоинт для чата: `POST /api/v1/chat`. Историю диалога хранит приложение и присылает её целиком с каждым сообщением; бэкенд сам оставляет из неё 10 последних сообщений.

Запрос:

```
{
  "message": "С мясом, и листы без варки",
  "history": [
    {"role": "user", "content": "Помоги с лазаньей"},
    {"role": "assistant", "content": "С удовольствием! Лазанья будет с мясом или овощная?"}
  ]
}
```

Успешный ответ (200):

```
{ "reply": "Отличный выбор! ..." }
```

После ответа приложение добавляет в историю две записи: вопрос пользователя (`role: user`) и ответ (`role: assistant`). Роль `system` присылать нельзя: запрос с ней получит ответ 422.

Код	Когда	Что показать пользователю
200	Ответ получен	Поле <code>reply</code>
422	Пустое сообщение, длиннее 2 000 символов, неверный формат <code>history</code>	Ошибка ввода; в норме не возникает, если приложение проверяет поле само

Код	Когда	Что показать пользователю
502	Провайдер вернул ошибку (неверный ключ или id модели, кончились деньги, лимит)	Поле <code>detail</code> — готовый текст
503	Сервер модели недоступен (LM Studio выключен, нет сети)	Поле <code>detail</code> — готовый текст
504	Модель не ответила за таймаут (120 с локально, 60 с в облаке)	Поле <code>detail</code> — готовый текст

Для приложения: таймаут HTTP-запроса — не меньше 2 минут, на время ожидания показывать индикатор «печатает...». Адрес бэкенда при разработке: Android-эмулятор — `http://10.0.2.2:8000`, настоящий телефон — IP ноутбука в той же Wi-Fi сети, например `http://192.168.1.5:8000`.

`GET /api/v1/health` возвращает `{"status", "llm", "provider", "model"}` — удобно для мониторинга и экрана «ИИ временно недоступен».

5. Настройки и переход в облако

Все настройки задаются переменными окружения: локально — в файле `.env`, на хостинге — в панели переменных окружения сервиса.

Переменная	Обязательна	Значение
<code>LLM_PROVIDER</code>	да	<code>lmstudio</code> , <code>openrouter</code> или <code>together</code>
<code>LLM_MODEL</code>	для LM Studio — да	<code>qwen/qwen3.5-9b</code> (LM Studio и OpenRouter), <code>Qwen/Qwen3.5-9B</code> (Together)
<code>LLM_API_KEY</code>	для облака — да	Ключ из личного кабинета провайдера
<code>LLM_BASE_URL</code>	нет	Свой адрес сервера, если он отличается от стандартного
<code>LLM_TIMEOUT_SECONDS</code>	нет	По умолчанию 120 (LM Studio) или 60 (облако)
<code>LLM_MAX_RETRIES</code>	нет	По умолчанию 0 (LM Studio) или 2 (облако)

Чтобы перейти в облако, достаточно поменять `.env`, например на OpenRouter:

```
LLM_PROVIDER=openrouter
LLM_MODEL=qwen/qwen3.5-9b
LLM_API_KEY=sk-or-...
```

Адрес провайдера и имена нестандартных параметров (`repeat_penalty` у LM Studio, `repetition_penalty` в облаке) клиент подставляет сам. Ошибка в имени провайдера или отсутствие ключа ловится при старте: сервер не запустится и напишет причину.

Ориентир по цене: при 120M токенов в месяц — около \$13 (OpenRouter) или \$22 (Together). Расчёт — в документе «ChefAI: inference-решение и расчёт стоимости».

6. Что осталось сделать бэкенду

Первые три пункта нужны для MVP, остальные — улучшения.

- ☐ **Подключить шлюз вежливости.** В начале функции `chat` в `main.py`, до вызова модели: `if not is_polite(body.message): return ChatResponse(reply="Извините, я не могу помочь с этим запросом. Пожалуйста, будьте вежливы.")`. Фраза совпадает с той, что выдаёт модель, чтобы поведение было одинаковым. Без шлюза каждое грубое сообщение стоит ~1 300 токенов и ~30 секунд работы модели.
- ☐ **Встроить модуль в общий бэкэнд.** Если у бэкенда уже есть своё FastAPI-приложение, перенести эндпоинты из `main.py` туда (например, через `APIRouter`) и создавать один `ChefLlmClient` на всё приложение при старте, а не на каждый запрос.
- ☐ **Задеплоить бэкэнд** на хостинг с работой 24/7 (Render, Railway или VPS; видеокарта не нужна) и переключить `LLM_PROVIDER` на облачного провайдера. Ключ — только в переменных окружения хостинга.
- ☐ Добавить `.env` в `.gitignore`.
- ☐ Ограничить частоту запросов на одного пользователя (например, 20 сообщений в минуту), чтобы один пользователь не мог израсходовать бюджет.
- ☐ Логировать расход токенов (клиент уже пишет его в лог) и раз в неделю сверять с прогнозом 120M токенов в месяц.
- ☐ Если приложению нужно, чтобы ответ появлялся постепенно, перейти на потоковую передачу (`stream=True`); сейчас ответ приходит целиком.

7. Ограничения и договорённости

- **Промпт и параметры генерации** (`SYSTEM_PROMPT` и `GenerationParams` в `chef_llm_client.py`) меняет только ML. Промпт проверен на трёх сценариях тона: обычный вопрос, требовательный тон (ответ + мягкая просьба о вежливости), мат (только фраза-отказ). Любая правка проверяется заново через `test_lm.py`.
- **Длина промпта** — около 1 300 токенов, он отправляется с каждым сообщением. Поэтому промпт нельзя расширять без необходимости: это прямо влияет на стоимость.
- **Ответы приходят обычным текстом**, без Markdown: так задано в промпте. Если экран чата научится показывать Markdown, строку о разметке в промпте можно убрать — сообщить ML.
- **Модель базовая**, без дообучения: поведение задано системным промптом. Качество отдельных ответов неидеально (иногда выдумывает детали, задаёт вопрос не в том месте) — примеры неудачных ответов собирать и передавать ML.
- **Модель не помнит пользователей между диалогами**: всё, что она знает о разговоре, — это история из запроса.